

Week 4 – Task 11 (EH)

How the Web Works + Burp Suite Intro

Field	Details
Submitted By	MANOJ
Role	Cyber Security Intern
Internship Organisation	TechBiz Security
Internship ID	TBS-INT-2026-518
Submitted Date	12 September 2026

Table of Contents

1. Introduction
2. HTTP Request Structure
3. HTTP Response Structure
4. GET vs POST
5. Cookies and Sessions
6. DVWA Lab Environment
7. DVWA Installation and Configuration
8. DVWA Database and Service Verification
9. DVWA Successful Access
10. Burp Suite Configuration
11. HTTP Request Interception with Burp Suite
12. Request Modification Concept
13. TryHackMe Resource Availability
14. Practical Learning Outcomes
15. Security Relevance
16. Challenges and Troubleshooting
17. Conclusion
18. Screenshot Evidence

Task Objective

The objective of this task was to learn the fundamentals of how web applications communicate through HTTP/HTTPS and to gain practical experience with Burp Suite as an intercepting proxy. The practical environment used Damn Vulnerable Web Application (DVWA) hosted locally on Kali Linux. This report covers HTTP request and response fundamentals, HTTP methods, headers, cookies and sessions, the configuration of a local DVWA lab, and the interception of a DVWA HTTP request using Burp Suite Professional.

1. Introduction

HTTP (Hypertext Transfer Protocol) is the application-layer protocol used by web browsers and web servers to exchange data. HTTPS is the encrypted form of HTTP, where traffic is secured using TLS to protect confidentiality and integrity between the client and server.

A web browser acts as the client, sending requests to a web server that hosts the application. The web server processes each request and returns a response, which the browser then renders. This client-server exchange, known as the request/response cycle, forms the foundation of every interaction with a web application.

Understanding HTTP is essential for web application security testing because nearly every attack against a web application — from parameter tampering to session hijacking — involves manipulating or analyzing the underlying HTTP requests and responses. Tools such as Burp Suite allow a tester to intercept, inspect, and modify this traffic before it reaches the server.

2. HTTP Request Structure

An HTTP request is composed of several key components:

- Request method — indicates the action being performed (e.g. GET, POST).
- Request URI/path — the resource being requested on the server.
- HTTP version — the protocol version used, typically HTTP/1.1.
- Host header — specifies the target domain or IP address.
- User-Agent header — identifies the client browser and operating system.
- Accept headers — indicate the content types the client can process.
- Cookie header — carries session and state information.
- Other headers — such as Referer, Connection, and Accept-Encoding.
- Request body — present in methods such as POST, containing submitted data.

The following request was captured using Burp Suite Professional while browsing the local DVWA application, and is used here as a practical example (session values shortened for confidentiality):

```
GET /DVWA/index.php HTTP/1.1
Host: 127.0.0.1
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
Referer: http://127.0.0.1/DVWA/login.php
Cookie: security=impossible; PHPSESSID=...(redacted)
```

In this request, the method GET indicates that the client is requesting a resource rather than submitting data. The Host header confirms the request is directed at the local DVWA instance (127.0.0.1). The User-Agent identifies the browser used, and the Referer header shows that the request originated from the DVWA login page. The Cookie

header carries the security level setting and the session identifier, which are explained further in Section 5. The session identifier value itself has been redacted in this report to avoid unnecessary exposure of live session data.

3. HTTP Response Structure

An HTTP response consists of a status code, response headers, and a response body. The status code indicates the outcome of the request, the headers provide metadata about the response (such as content type and server information), and the body contains the actual content returned to the client, such as HTML, JSON, or an error message.

Common HTTP status codes encountered during web application testing include:

Status Code	Meaning
200 OK	The request was processed successfully.
301 / 302 Redirect	The client is redirected to a different resource.
400 Bad Request	The request was malformed or could not be understood.
401 Unauthorized	Authentication is required to access the resource.
403 Forbidden	The server understood the request but refuses to authorize it.
404 Not Found	The requested resource does not exist on the server.
500 Internal Server Error	The server encountered an unexpected error.

Status codes are valuable during web application security testing because they reveal how an application responds to both valid and invalid input. For example, unexpected 500 errors may indicate that an application is mishandling malformed data, while 403 or 401 responses can help identify access-control boundaries that may be worth testing further.

4. GET vs POST

GET and POST are the two most commonly used HTTP methods in web applications. They differ in how parameters are transmitted and in their typical use cases, as summarized below.

Aspect	GET	POST
Location of parameters	Appended to the URL as a query string	Sent within the request body
Visibility in URL	Visible and can appear in browser history/logs	Not visible in the URL
Typical use	Retrieving data (e.g. loading a page)	Submitting data (e.g. login forms)
Request body	Typically absent	Present, containing form or API data
Bookmarking / caching	Can be bookmarked and cached	Not bookmarked or cached by

Aspect	GET	POST
		default
Security considerations	Sensitive data should not be sent via GET, as it may be logged or cached	Still visible to an intercepting proxy; must be combined with HTTPS and proper server-side validation

A simple GET example used conceptually to illustrate query parameters:

```
GET /DVWA/index.php?test=burp HTTP/1.1
```

A conceptual POST example, illustrating how login credentials are typically submitted in the request body:

```
POST /DVWA/login.php HTTP/1.1
```

```
username=admin&password=password
```

This POST example is provided for conceptual understanding only. No POST request was captured or intercepted during this practical exercise; only the GET request described in Section 11 was demonstrated with screenshot evidence.

5. Cookies and Sessions

Cookies are small pieces of data stored by the browser and sent back to the server with subsequent requests. They allow a web application to maintain state across an inherently stateless protocol such as HTTP.

In the intercepted DVWA request, the Cookie header contained a PHPSESSID value. PHPSESSID is the default session identifier used by PHP applications; it associates a given browser session with server-side session data, such as the logged-in user and the configured DVWA security level. The presence of this cookie in the captured request (Figure 5) confirms that DVWA was maintaining an authenticated session for the browser used during testing.

Session cookies are important during web application security testing because they are frequently targeted in attacks such as session hijacking, session fixation, and cross-site scripting. For this reason, session identifiers and other sensitive cookie values have been redacted in this report rather than reproduced in full, in line with responsible handling of security-relevant data.

6. DVWA Lab Environment

Component	Details
Operating System	Kali Linux
Web Application	Damn Vulnerable Web Application (DVWA)
Web Server	Nginx
Database	MariaDB
PHP	PHP 8.4 with PHP-FPM

Component	Details
Web Application URL	http://127.0.0.1/DVWA/

DVWA was deployed locally inside the Kali Linux virtual machine as an intentionally vulnerable application intended solely for authorized cybersecurity training. All testing described in this report was performed against this local, isolated instance.

7. DVWA Installation and Configuration

The DVWA environment was set up in the following chronological steps:

1. DVWA was cloned into /var/www/html/DVWA using Git.
2. Required PHP, database, and supporting packages were installed.
3. MariaDB was started and enabled to run at boot.
4. A DVWA database (dvwa) and a corresponding database user (dvwa) were configured.
5. The DVWA configuration file was created from config.inc.php.dist.
6. Nginx was configured to serve PHP content through PHP 8.4-FPM.
7. PHP 8.4-FPM was verified as active and running.
8. DVWA's setup.php page was accessed, and the database was created/reset.
9. DVWA was successfully accessed through the local browser.

The default DVWA administrative credentials were used to log in to the isolated lab environment; no credential values are reproduced in this report.

8. DVWA Database and Service Verification

```

kali@kali:~$ cd /var/www/html
kali@kali:~/var/www/html$ git clone https://github.com/digininja/DVWA.git
Cloning into 'DVWA'...
remote: Enumerating objects: 5885, done.
remote: Counting objects: 100% (26/26), done.
remote: Compressing objects: 100% (16/16), done.
remote: Total 5885 (delta 13), reused 11 (delta 10), pack-reused 5779 (from 2)
Receiving objects: 100% (5885/5885), 2.94 MiB | 337.00 KiB/s, done.
Resolving deltas: 100% (2874/2874), done.
kali@kali:~/var/www/html$ sudo apt update
Get:1 https://packages.microsoft.com/repos/code stable/main amd64 Packages [30.3 kB]
Hit:2 https://packages.microsoft.com/repos/code stable/main amd64 Packages [30.3 kB]
Hit:3 https://apt.kali.org/archive-keyring kali-rolling/main amd64 Packages [14.8 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.3 MB]
Get:5 http://kali.download/kali kali-rolling/main amd64 Contents (deb) [52.7 MB]
Get:6 http://kali.download/kali kali-rolling/contrib amd64 Packages [113 kB]
Get:7 http://kali.download/kali kali-rolling/contrib amd64 Contents (deb) [261 kB]
Get:8 http://kali.download/kali kali-rolling/non-free amd64 Packages [220 kB]
Get:9 http://kali.download/kali kali-rolling/non-free amd64 Contents (deb) [915 kB]
Get:10 http://kali.download/kali kali-rolling/non-free-firmware amd64 Packages [16.0 kB]
Get:11 http://kali.download/kali kali-rolling/non-free-firmware amd64 Contents (deb) [48.6 kB]
Fetched 76.8 MB in 14s (5,618 kB/s)
985 packages can be upgraded. Run 'apt list --upgradable' to see them.
Note: selecting 'php8.4-mysql' instead of 'php-mysql'
php is already the newest version (8.4.4-1).
mariadb-server is already the newest version (1:11.6.0-1).
mariadb-server set to manually installed.
git is already the newest version (1:2.53.0-1).
The following packages were automatically installed and are no longer required:
  blueschiffm network-manager-gnome
Use 'sudo apt autoremove' to remove them.
Upgrading:
  libapache2-mod-php8.4 php8.4 php8.4-cli php8.4-common php8.4-mysql php8.4-opcache php8.4-readline
Installing:
  php-curl php-gd php-mbstring php-xml
Installing dependencies:
  
```

Figure 1: DVWA database configuration and MariaDB database setup.

This screenshot provides evidence of the database configuration steps required for DVWA, including the creation of the dvwa database, the creation of the dvwa database user, and the granting of privileges, followed by configuration of the DVWA config file and verification of the Nginx service.

```

manuarya@kali: ~/var/www/html
$ sudo systemctl start mariadb

manuarya@kali: ~/var/www/html
$ sudo systemctl enable mariadb

manuarya@kali: ~/var/www/html
$ sudo systemctl status mariadb
● mariadb.service - MariaDB 11.8.8 database server
   Loaded: loaded (/usr/lib/systemd/system/mariadb.service; enabled; preset: disabled)
   Active: active (running) since Sat 2020-09-12 12:20:00 IST; 1h 35min ago
   Invocation: 957dab3fd247c9e0678ae1e207096
   Docs: man:mariadb(8)
        https://mariadb.com/kb/en/library/systemd/
   Main PID: 800 (mariadbd)
   Status: "Taking your SQL requests now..."
   Tasks: 16 (limit: 4376)
   Memory: 172.9M (peak: 177.3M)
   CPU: 0.964s
   CGroup: /system.slice/mariadb.service
           └─800 /usr/sbin/mariadbd

Sep 12 12:19:55 kali mariadbd[800]: 2020-09-12 12:19:55 0 [Note] Recovering after a crash using tc.log
Sep 12 12:19:55 kali mariadbd[800]: 2020-09-12 12:19:55 0 [Note] Starting table crash recovery...
Sep 12 12:19:55 kali mariadbd[800]: 2020-09-12 12:19:55 0 [Note] Crash table recovery finished.
Sep 12 12:19:59 kali mariadbd[800]: 2020-09-12 12:19:59 0 [Note] Server socket created on IP: '127.0.0.1', port: '3306'.
Sep 12 12:19:59 kali mariadbd[800]: 2020-09-12 12:19:59 0 [Note] mariadbd: Event Scheduler: Loaded 0 events
Sep 12 12:19:59 kali mariadbd[800]: 2020-09-12 12:19:59 0 [Note] /usr/sbin/mariadbd: ready for connections.
Sep 12 12:19:59 kali mariadbd[800]: Version: '11.8.8-MariaDB-1 from Debian' socket: '/run/mysqld/mysqld.sock' port: 3306 -- Please help get to 10k stars at https://github.com/MariaDB/Server
Sep 12 12:20:00 kali systemd[1]: Started mariadb.service - MariaDB 11.8.8 database server.
Sep 12 12:20:00 kali /etc/mysql/debian-start[999]: Upgrading MariaDB tables, if necessary.
Sep 12 12:20:00 kali /etc/mysql/debian-start[999]: Checking for insecure root accounts.

manuarya@kali: ~/var/www/html
$

```

Figure 2: MariaDB service running successfully on Kali Linux.

MariaDB stores all application data used by DVWA, including user accounts and challenge-specific data. This screenshot confirms that the MariaDB service was active and accepting connections, which was a prerequisite for DVWA to initialize its database correctly.

```

manuarya@kali: ~/var/www/html/DVWA/config
$ sudo systemctl start nginx

manuarya@kali: ~/var/www/html/DVWA/config
$ sudo systemctl enable nginx

manuarya@kali: ~/var/www/html/DVWA/config
$ sudo systemctl status nginx
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/usr/lib/systemd/system/nginx.service; disabled; preset: disabled)
   Active: active (running) since Sat 2020-09-12 14:00:57 IST; 6s ago
   Invocation: 96a2db64720446ba207cc0c1a82096a1
   Docs: man:nginx(8)
   Process: 16307 ExecStartPre=/usr/sbin/nginx -t -q -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Process: 16308 ExecStart=/usr/sbin/nginx -g daemon on; master_process on; (code=exited, status=0/SUCCESS)
   Main PID: 16308 (nginx)
   Tasks: 4 (limit: 6934)
   Memory: 5.7M (peak: 6M)
   CPU: 7ms
   CGroup: /system.slice/nginx.service
           └─16308 "nginx: master process /usr/sbin/nginx -g daemon on; master_process on;"
             └─16391 "nginx: worker process"
               └─16392 "nginx: worker process"
                 └─16393 "nginx: worker process"

Sep 12 14:00:57 kali systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server.
Sep 12 14:00:57 kali systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.

manuarya@kali: ~/var/www/html/DVWA/config
$

```

Figure 3: Nginx web server running successfully.

Nginx was configured as the web server for the local DVWA deployment, responsible for serving DVWA's PHP content by forwarding requests to PHP-FPM. This screenshot confirms that the Nginx service was active and running after configuration.

9. DVWA Successful Access

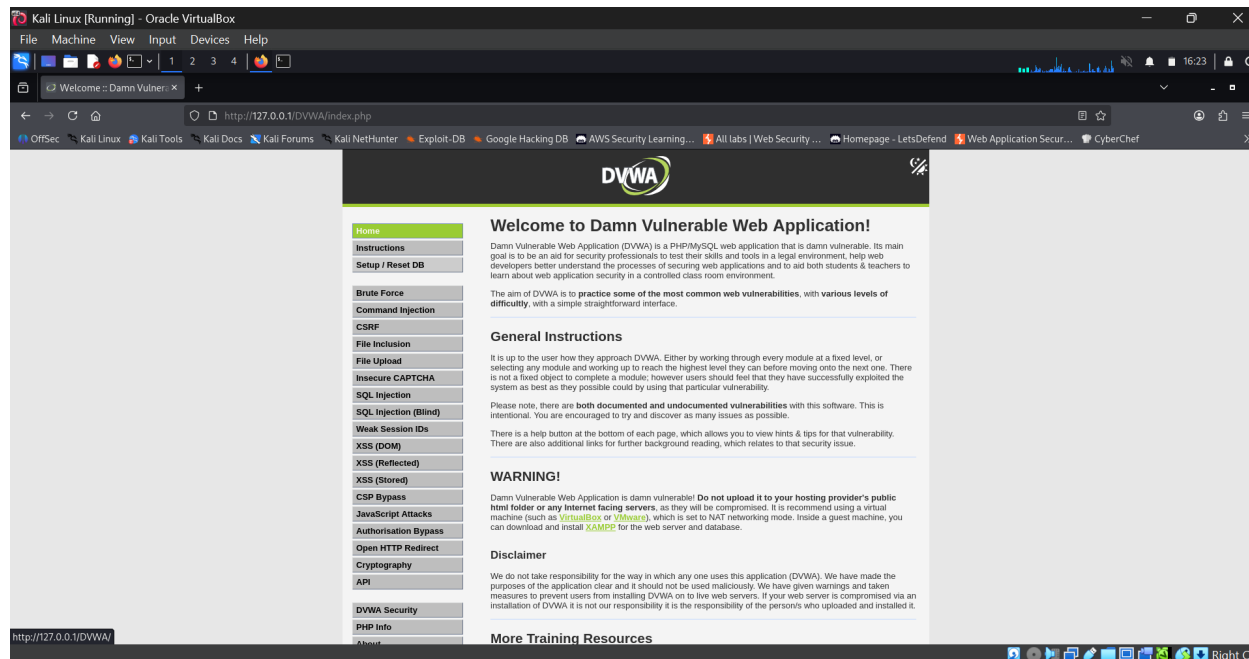


Figure 4: Successfully authenticated DVWA homepage.

DVWA was successfully initialized and accessed through the local browser, confirming that the web application environment was fully operational. The application was hosted locally at 127.0.0.1 and was used exclusively for authorized security training within the isolated Kali Linux virtual machine.

10. Burp Suite Configuration

Burp Suite Professional was used as the intercepting proxy for this task. Burp Suite sits between the browser and the target web application, allowing every HTTP request and response to be captured, inspected, and, where required, modified before it continues to its destination.

The configuration used for this practical exercise was as follows:

- Browser traffic was routed through the Burp Suite proxy.
- The proxy listener used the local address 127.0.0.1:8080.
- Proxy interception was enabled in Burp Suite.
- A request to the local DVWA application was captured.

The resulting traffic flow for this configuration was:

```
Browser -> Burp Suite Proxy -> DVWA Web Server
DVWA Web Server -> Burp Suite Proxy -> Browser
```

11. HTTP Request Interception with Burp Suite

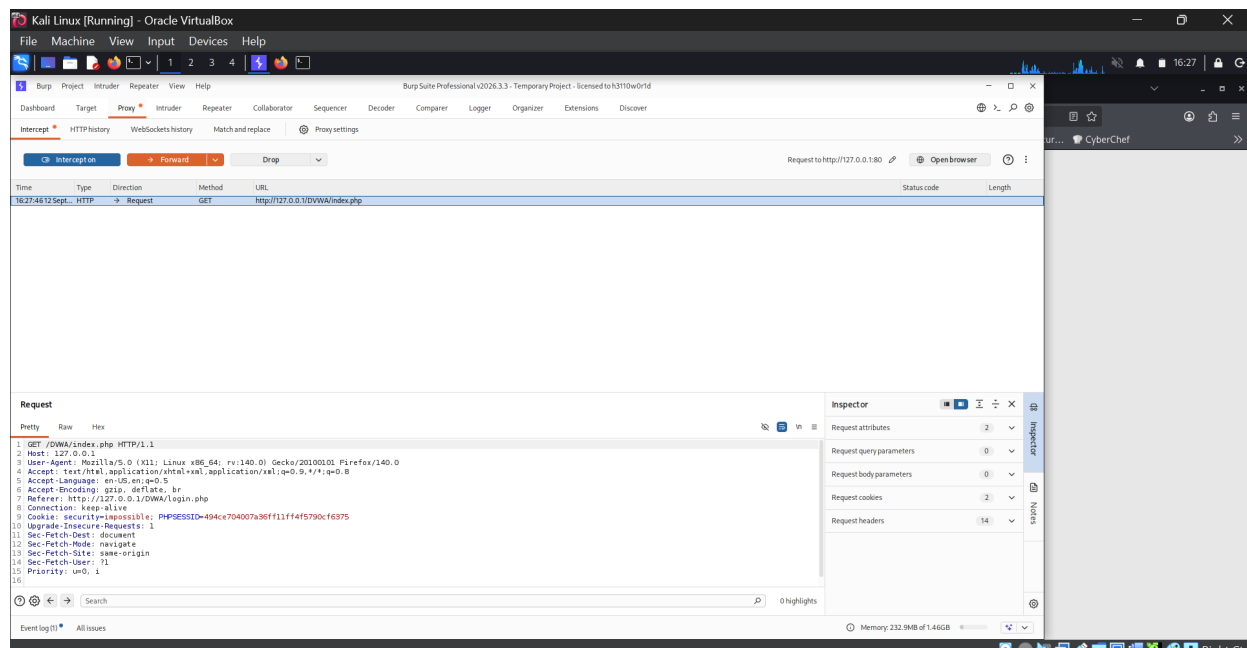


Figure 5: GET request intercepted using Burp Suite Professional.

The screenshot shows the Burp Suite Professional Proxy/Intercept interface with the following details visible:

- Intercept is enabled, and the captured request is held before forwarding.
- A GET request directed at the local DVWA application.
- Request path: `/DVWA/index.php`
- Host: `127.0.0.1`
- Standard HTTP request headers, including User-Agent, Accept, and Referer.
- Cookie/session information, including the security level and PHPSESSID values.

This demonstrates that Burp Suite allowed the HTTP request to be inspected before it was forwarded to the web server. Only the interception and inspection of this request was performed and evidenced; no modified request was forwarded as part of this exercise.

12. Request Modification Concept

A key capability of an intercepting proxy in a controlled web security lab is the ability to modify a captured HTTP request before it is forwarded to the server. This is typically used to test how an application responds to altered input, such as additional or manipulated parameters.

A harmless conceptual example of such a modification would be adding a benign query parameter to the intercepted request, for instance:

```
GET /DVWA/index.php?test=burp HTTP/1.1
```

This example illustrates, at a conceptual level, how an intercepted request could be edited within Burp Suite before being forwarded. It is important to distinguish this from what was actually demonstrated in the practical exercise: the practical work captured and inspected the unmodified GET request shown in Figure 5. No modified

request, including the example above, was forwarded to the server or evidenced by screenshot, and this report does not claim that such a modification was tested.

13. TryHackMe Resource Availability

The task instructions referenced two TryHackMe rooms as learning resources: "Burp Suite: The Basics" and "Walking An Application". Upon attempting to access these rooms, both were found to be restricted to TryHackMe Premium subscribers and were not available under the current Free-tier account access.

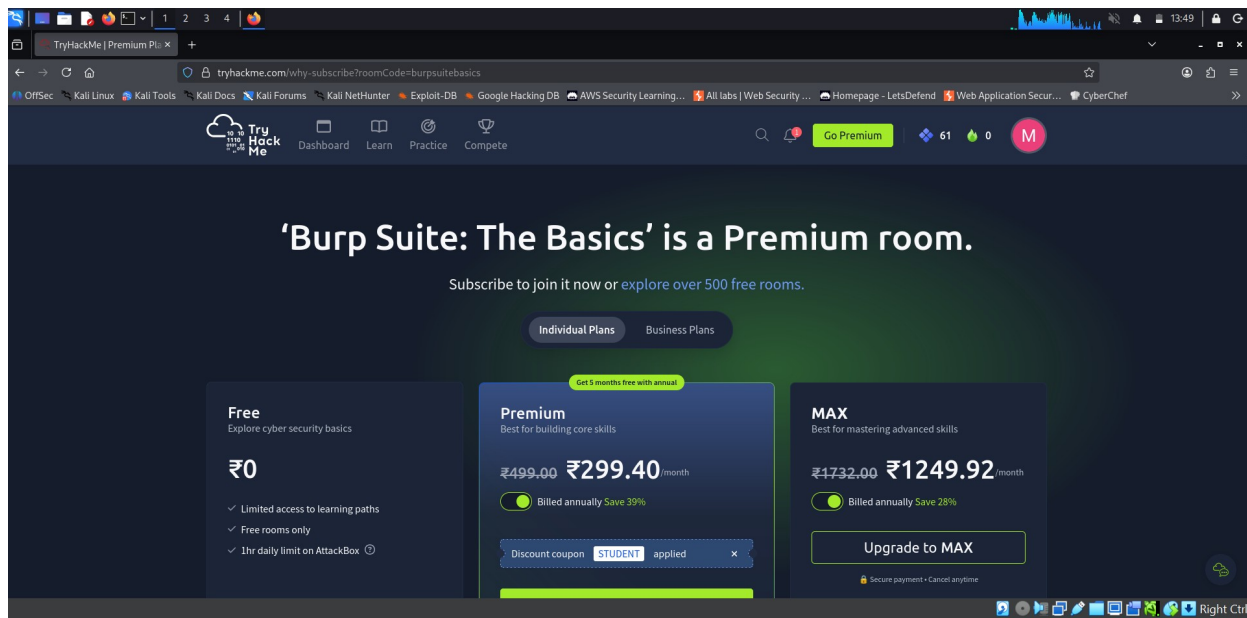


Figure 6: TryHackMe "Burp Suite: The Basics" identified as a Premium room.

This screenshot confirms that the "Burp Suite: The Basics" room requires a Premium subscription and was therefore unavailable under the account's current Free access level.

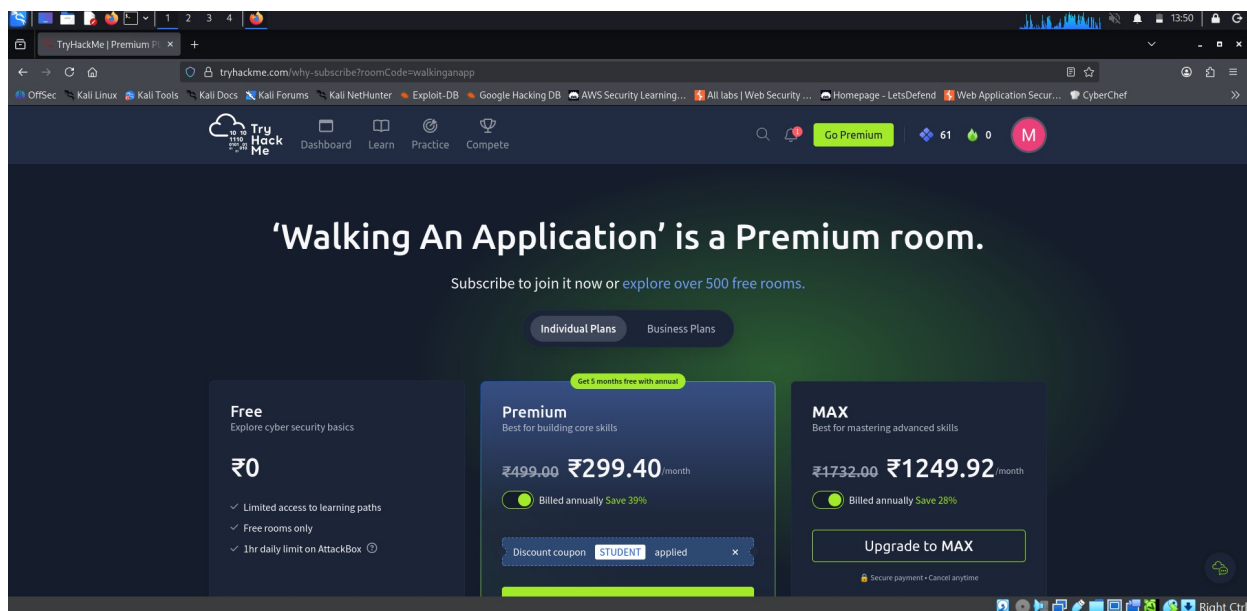


Figure 7: TryHackMe "Walking An Application" identified as a Premium room.

This screenshot confirms that the "Walking An Application" room was also identified as Premium-only and could not be accessed.

Because neither specified room could honestly be reported as completed, the practical requirement for this task was addressed instead through the local DVWA and Burp Suite lab environment described in the preceding sections. Completion of the Premium TryHackMe rooms is not claimed in this report.

14. Practical Learning Outcomes

- Understanding the HTTP request/response lifecycle.
- Identifying and interpreting GET requests.
- Understanding the purpose of common HTTP headers.
- Understanding cookies and session management using PHPSESSID.
- Configuring a local web application environment from source.
- Working with Nginx and PHP-FPM to serve a PHP application.
- Configuring and administering a MariaDB database.
- Configuring DVWA, including its database and application settings.
- Configuring Burp Suite Professional as an intercepting proxy.
- Capturing and inspecting live web requests using Burp Suite.
- Understanding, at a conceptual level, how HTTP requests can be modified in an authorized security testing environment.
- Recognizing the practical differences between GET and POST requests.

15. Security Relevance

The skills demonstrated in this task are directly relevant to several areas of cybersecurity practice:

- Web application penetration testing — analyzing and manipulating HTTP traffic to identify vulnerabilities.
- Vulnerability Assessment and Penetration Testing (VAPT) — understanding request/response behavior as part of structured assessments.
- SOC and web traffic investigation — recognizing normal versus anomalous HTTP traffic patterns.
- Bug bounty methodology — using proxy tools to explore application behavior and identify potential issues.
- Secure application development — understanding how requests are constructed helps developers build more secure input handling.
- Manual HTTP request analysis — a foundational skill for nearly all forms of web security testing.
- Session security testing — evaluating how session cookies are issued, transmitted, and protected.
- Request manipulation testing — understanding how intercepted requests can be altered to test application logic and validation.

16. Challenges and Troubleshooting

10. DVWA initially required manual configuration of the local web server and database before it could be accessed, as no environment was pre-configured.

11. Nginx initially returned a 403 Forbidden response when attempting to access DVWA, indicating a configuration issue rather than a missing resource.
12. Investigation showed that Nginx required proper PHP-FPM configuration, as the PHP location block in the site configuration was initially commented out.
13. PHP 8.4-FPM was installed and configured to resolve this issue.
14. Nginx was updated to use the PHP-FPM socket at /run/php/php8.4-fpm.sock.
15. Following this change, the DVWA setup page was successfully accessed and the database was initialized.
16. The TryHackMe rooms specified in the task instructions were found to be Premium-only; as a result, the practical work was continued and evidenced using the local DVWA and Burp Suite lab environment instead.

17. Conclusion

This task provided practical exposure to the fundamentals of web application communication and to Burp Suite as an intercepting proxy. The local DVWA environment, deployed on Nginx, PHP 8.4-FPM, and MariaDB, was successfully configured and accessed, and a live HTTP GET request was captured and inspected using Burp Suite Professional. This exercise also provided hands-on experience with HTTP headers, cookies, and session identifiers, and reinforced the conceptual distinction between GET and POST requests.

The TryHackMe rooms "Burp Suite: The Basics" and "Walking An Application" referenced in the task instructions were found to be Premium-only and were not completed as part of this exercise. The practical requirement was instead fulfilled through the local DVWA and Burp Suite lab, as documented in this report.

18. Screenshot Evidence

No.	Screenshot	Description
1	01_dvwa_database_configured.png	DVWA database configuration
2	02_mariadb_service_running.png	MariaDB service running
3	03_nginx_service_running.png	Nginx service running
4	04_dvwa_logged_in_homepage.png	DVWA successfully accessed
5	05_burp_dvwa_get_request_intercepted.png	HTTP GET request intercepted in Burp
6	06_tryhackme_burp_suite_basics_premium.png	TryHackMe Burp Suite room shown as Premium
7	07_tryhackme_walking_application_premium.png	TryHackMe Walking An Application room shown as Premium